

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE FÍSICA DE SÃO CARLOS

NICOLAS OLIVEIRA ROSSONI

Redes neurais aplicadas na física: usando PINN e MixFunn para
resolver EDPs

São Carlos

2026

NICOLAS OLIVEIRA ROSSONI

Redes neurais aplicadas na física: usando PINN e MixFunn para resolver EDPs

Trabalho de Conclusão de Curso apresentado ao Instituto de Física de São Carlos da Universidade de São Paulo para obtenção do título de Bacharel em Física.

Orientador: Prof. Dr. Celso Jorge Villas-Boas — UFS-Car

São Carlos

2026

RESUMO

O uso de redes neurais para resolver equações diferenciais parciais (EDPs) é uma linha de pesquisa em desenvolvimento ativo, sustentada por avanços recentes em *hardware* e algoritmos computacionais — em particular a diferenciação automática como ferramenta nativa — que viabilizam representar tais soluções de forma compacta e geral. A arquitetura de rede mais comum nessa abordagem é a *multi-layer perceptron* (MLP); sobre ela, a *Physics-Informed Neural Network* (PINN) é um método de treino que incorpora a própria EDP na função de perda, e a *Mix2Funn* (MixFunn) é um modelo alternativo de neurônio que pode substituir o neurônio tradicional da MLP. Daqui em diante, ao mencionarmos PINN, estamos nos referindo à MLP treinada com esse método. Aplicamos ambos a problemas físicos diversos para validar os elementos centrais do paradigma. Primeiramente, o escoamento de Kovasznay — caso particular das equações de Navier–Stokes com solução analítica fechada — foi usado como base para um estudo aprofundado: variação de arquitetura, comparação entre os regimes supervisionado e não-supervisionado, análise do decaimento da loss, e métodos de regularização (*dropout* e subamostragem). Em seguida, ambas as redes foram aplicadas ao escoamento sanguíneo em um aneurisma intracraniano tridimensional, usando dados de paciente real do *dataset* ANEUMO, comparando os regimes supervisionado e não-supervisionado em um problema de geometria complexa. Por fim, dois casos complementares (Schrödinger não-linear e magnetostática 2D com interface dielétrica) foram apresentados como demonstração condensada da diversidade de EDPs cobertas. Como principais resultados, ambas as arquiteturas reproduziram fielmente as soluções de referência em todos os problemas estudados, com a MixFunn alcançando desempenho competitivo usando arquiteturas substancialmente menores; as principais dificuldades enfrentadas foram a forte sensibilidade dos métodos aos hiperparâmetros e a tendência dos métodos a colapsarem em soluções triviais. Conclui-se que ambas as redes são ferramentas viáveis para resolução de EDPs, com perfis complementares: a PINN (MLP) oferece capacidade expressiva universal, e a MixFunn traz interpretabilidade e simplicidade. Cada problema demanda exploração própria de seu espaço de soluções e hiperparâmetros, em contraste com a execução determinística dos métodos numéricos tradicionais.

Palavras-chave: EDP. PINN. MixFunn. Deep Learning. Navier–Stokes.

1 INTRODUÇÃO

Objetivos

O objetivo deste trabalho se divide em duas frentes. A primeira é exploratória: estudar como redes neurais podem ser empregadas para resolver equações diferenciais parciais (EDPs) em física, entender como funcionam, em que regimes convergem, e o que cada arquitetura revela sobre a estrutura do problema. Sobre essa base, a segunda frente é comparativa: avaliar dois métodos específicos, a *Physics-Informed Neural Network* (PINN) [Raissi 2019], e a *Mix2Funn* [Farias 2025], aplicados na

multilayer perceptron (MLP) [LeCun 2015], em uma sequência de problemas que vão da mecânica clássica à mecânica quântica, comparando com a solução analítica dos problemas ou com métodos numéricos tradicionais, como o de Runge-Kutta.

1.1 O que é uma rede neural (MLP)

Redes neurais [LeCun 2015] são métodos com estrutura complexa e existem em vários tipos. O foco deste trabalho é a MLP, ou *multilayer perceptron*, que é um tipo de rede neural.

As MLPs, em geral, são algoritmos que aplicam uma transformação em um conjunto de dados de entrada (*input*), podendo variar a sua dimensionalidade na saída (*output*). Elas têm a liberdade para manipular a entrada com funções lineares e não lineares, correlacionando cada informação entre si.

A MLP possui várias camadas e diversos parâmetros que modificam essa transformação. Dependendo da estrutura da rede e dos parâmetros escolhidos, ela se torna um “aproximador universal” [Hornik 1989] de qualquer transformação desejada. Quanto maior a liberdade da rede para modificar os dados (mais parâmetros e camadas), maior sua expressividade.

Para uma dada “transformação ideal”, existe um conjunto de parâmetros capaz de aproximar esse comportamento, para um dado erro, se a rede for expressiva o suficiente. Para resolver o problema, precisamos de um processo de otimização que encontre esses parâmetros — essa capacidade de aproximação fica, na prática, restrita pelo processo. A partir disso, podemos usar métodos como o *back-propagation* combinado com gradiente descendente para encontrar esses parâmetros; existem também outras estratégias de otimização para redes neurais, como algoritmo genético [Goldberg 1988] e *simulated annealing* [Kirkpatrick 1983]. O treinamento pode ser feito de forma supervisionada, em que a rede aprende a partir de rótulos — pares de entrada e saída esperada conhecidos previamente —, ou de forma não supervisionada, em que a rede aprende sem rótulos.

A MLP é um exemplo clássico de rede neural, uma rede totalmente conectada que organiza essa transformação em camadas sucessivas (cada camada é um vetor $\mathbf{h}_l$, como na Figura 1). Cada neurônio (cada componente $h_j^{(l)}$ de $\mathbf{h}_l$) de uma camada multiplica a entrada por seus próprios pesos ($W_{ji}^{(l)}$ é o peso que conecta o neurônio $h_i^{(l)}$ ao neurônio $h_j^{(l+1)}$, conforme as equações abaixo), soma um viés ($b_j^{(l)}$, número somado em cada neurônio para aumentar a expressividade da rede) e aplica uma função de ativação não-linear; a saída de uma camada é a entrada da seguinte, como mostra o diagrama da Figura 1.

A escolha da função de ativação σ é livre. Na prática, ela é um hiperparâmetro e existem duas restrições. Ela precisa ter derivada bem definida, para que possamos aplicar o *backpropagation*, explicado mais adiante; e precisa ser não-linear, pois caso contrário a rede inteira se reduziria a uma única transformação linear.

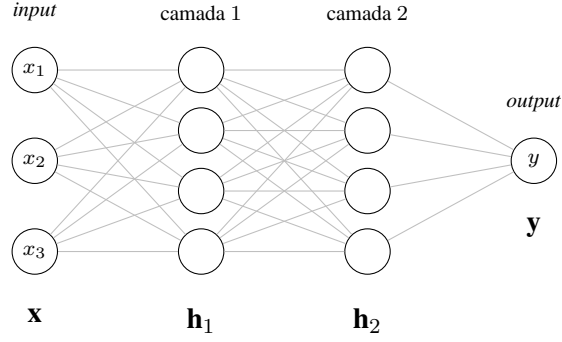


Figura 1: Diagrama de uma MLP de duas camadas ocultas. Cada aresta carrega um peso $W_{ji}^{(l)}$; cada nó das camadas ocultas soma um viés $b_j^{(l)}$ e aplica σ , que chamamos de função de ativação.

Componente a componente, a saída do j -ésimo neurônio da camada $l + 1$ é

$$h_j^{(l+1)} = \sigma \left(\sum_{i=1}^{n_l} W_{ji}^{(l)} h_i^{(l)} + b_j^{(l)} \right), \quad l = 0, 1, \dots, L - 1, \quad (1)$$

com $h_i^{(0)} = x_i$ e saída final $y_k = \sum_j W_{kj}^{(L)} h_j^{(L)} + b_k^{(L)}$. Empilhando os pesos $W_{ji}^{(l)}$ em uma matriz W_l e os vieses $b_j^{(l)}$ em um vetor $\mathbf{b}_l$, a equação (1) fica na sua forma vetorial:

$$\mathbf{h}_0 = \mathbf{x}, \quad \mathbf{h}_{l+1} = \sigma(W_l \mathbf{h}_l + \mathbf{b}_l), \quad \mathbf{y} = W_L \mathbf{h}_L + \mathbf{b}_L. \quad (2)$$

Agora com a estrutura da MLP deduzida, basta definir o número de camadas e o número de neurônios por camada, e treinar a rede. Treinar a rede é um problema de busca: queremos encontrar o conjunto de parâmetros que melhor aproxima a transformação-alvo. O método canônico parte da função de perda (“*loss*”), que é a maneira de quantificar o erro da rede. No caso supervisionado mais comum, temos um conjunto de N pares $(\mathbf{x}_i, \mathbf{y}_i)$, em que $\mathbf{x}_i$ é a entrada da rede e $\mathbf{y}_i$ é a saída esperada que conhecemos previamente. Queremos que a rede u_θ reproduza esses pares, e a perda é o erro quadrático médio:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|u_\theta(\mathbf{x}_i) - \mathbf{y}_i\|^2. \quad (3)$$

Como a perda é uma função explícita de cada parâmetro $\theta_j \in \{W_l, \mathbf{b}_l\}$, conseguimos calcular o gradiente $\partial \mathcal{L} / \partial \theta_j$, que mede quanto a perda muda quando aquele parâmetro varia. O *backpropagation* é o algoritmo que aplica a regra da cadeia ao longo das camadas para obter esse gradiente. Com o gradiente, sabemos a direção em que cada parâmetro deve variar; variamos cada parâmetro nessa direção, com magnitude controlada pela “*learning rate*” η , que define o quão rápido ou devagar é o aprendizado. Um η muito alto pode fazer a otimização ultrapassar o mínimo local; um η muito baixo

torna o treinamento mais lento. A atualização segue a equação abaixo:

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}. \quad (4)$$

Iterando esse passo, a perda decresce e os parâmetros se aproximam de um conjunto que aproxima a transformação desejada.

Esse é o método do gradiente descendente puro, base conceitual a partir da qual otimizadores mais sofisticados foram desenvolvidos. Otimizadores modernos como o *Adam* [Kingma 2014] ajustam dinamicamente a magnitude do passo combinando o histórico recente do gradiente (momento) com escala adaptativa por parâmetro, o que torna o treinamento mais rápido e robusto. Neste trabalho, usamos o Adam como otimizador padrão.

Está descrito assim o funcionamento básico das MLPs. Existem outras arquiteturas [Miranda 2025] em uso na ciência de dados moderna, como convolucionais, recorrentes e *transformers*, entre outras; mas a que adotaremos neste trabalho é uma reformulação específica da MLP para resolver EDPs: a PINN. A MLP é especialmente adequada para essa formulação devido à sua simplicidade e diferenciabilidade: permite calcular facilmente, via diferenciação automática, as derivadas da saída em relação às entradas que aparecem no resíduo da EDP.

1.2 Como redes neurais resolvem EDPs (PINN)

Neste trabalho, a “transformação ideal” a ser aproximada não vem apenas de dados experimentais, mas de uma EDP, de modo que a rede aprenda a solução. O objetivo é obter soluções como as dos métodos numéricos tradicionais, como o de Runge-Kutta [Butcher 1996] — método iterativo que aproxima a solução de equações diferenciais em cada passo a partir de avaliações ponderadas da derivada em pontos intermediários —, ou como métodos analíticos.

Com a infraestrutura das MLPs, obter $\partial u / \partial x$ ou $\partial^2 u / \partial t^2$ a partir de uma rede $u_\theta(x, t)$ é direto, o que viabiliza usar redes neurais como soluções de EDPs. Suponha que a rede $u_\theta(x, t)$ receba o ponto (x, t) como entrada e devolve o valor da função u como saída, conforme o diagrama da Figura 2.

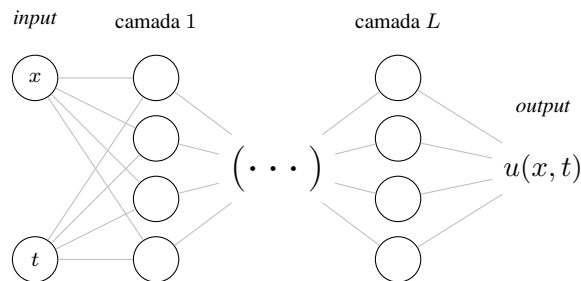


Figura 2: Rede neural que tenta aproximar a solução de uma EDP como uma função $u(x, t)$: as entradas x e t alimentam L camadas ocultas e a saída devolve o valor da função no ponto. A profundidade da rede é livre, indicada por $(\dots)$.

Por diferenciação automática, obtemos $\partial u/\partial x$, $\partial u/\partial t$, $\partial^2 u/\partial x^2$ e assim por diante, em qualquer ordem, em qualquer ponto do domínio, sem precisar de uma malha discreta — ao contrário dos métodos numéricos tradicionais, que dependem de discretização espacial e temporal. A partir disso, fica claro que conseguimos construir a função de perda em função das próprias derivadas: se a EDP é $\mathcal{F}[u] = 0$ no interior do domínio, com condição inicial $u(x, 0) = u_0(x)$ e condição de contorno $u(x, t) = g(x, t)$ na borda, definimos

$$\mathcal{L}(\theta) = \underbrace{\sum_{i=1}^{N_r} \frac{|\mathcal{F}[u_\theta](x_i, t_i)|^2}{N_r}}_{\mathcal{L}_{\text{res}}} + \underbrace{\sum_{i=1}^{N_{\text{CI}}} \frac{|u_\theta(x_i, 0) - u_0(x_i)|^2}{N_{\text{CI}}}}_{\mathcal{L}_{\text{CI}}} + \underbrace{\sum_{i=1}^{N_{\text{CC}}} \frac{|u_\theta(x_i, t_i) - g(x_i, t_i)|^2}{N_{\text{CC}}}}_{\mathcal{L}_{\text{CC}}}, \quad (5)$$

onde os pontos (x_i, t_i) são amostrados aleatoriamente via hipercubo latino dentro do domínio da função — a região do espaço onde queremos que a rede aproxime a solução —, respectivamente no interior, na condição inicial e na condição de contorno. Minimizando $\mathcal{L}$ pelo procedimento da seção anterior, a rede converge para algum mínimo local da loss — que pode ser a solução desejada (uma função que satisfaz a EDP e as condições) ou uma solução trivial. Métodos de regularização, tratados mais adiante, ajudam a fugir desses mínimos triviais.

Nessa formulação, o treino é não-supervisionado: não precisamos de uma solução numérica de referência nem de medições experimentais. A rede aprende a partir do próprio enunciado do problema, a EDP e suas condições inicial e de contorno são toda a informação necessária. Essa é a essência da *Physics-Informed Neural Network* (PINN), introduzida em [Raissi 2019].

Uma vez treinada, a rede u_θ é uma representação contínua da solução em todo o domínio: cada avaliação requer uma passagem direta pelo grafo. Métodos numéricos clássicos precisam discretizar o domínio em uma malha de antemão, e o custo escala linearmente com o número de pontos da grade — leva várias ordens de magnitude a mais que uma consulta à rede para os problemas tratados neste trabalho, já que os métodos numéricos clássicos precisam resolver todos os pontos da malha de uma vez, enquanto a rede pode ser consultada pontualmente em qualquer coordenada. A discrepância piora em domínios bidimensionais e tridimensionais, em que o número de pontos cresce com a potência da dimensão. A PINN (MLP) troca um custo alto de treino, único, por consultas baratas e pontuais durante o uso.

1.3 O que são MixFunns

A MLP da PINN é uma arquitetura “cega”: como suas ativações são funções genéricas (tipicamente tanh), a rede precisa aproximar, camada a camada, funções comuns em soluções da física (senos, exponenciais, polinômios). Isso pesa duas vezes: muitos parâmetros para uma estrutura simples, e a função final fica espalhada em milhares de pesos sem significado individual. A *Mix2Funn* [Farias 2025] reformula o neurônio com dois ganhos em mente. Primeiro, “viés indutivo”: em vez de redescobrir

funções, a rede recebe um conjunto fixo de funções elementares como conhecimento prévio do tipo de solução esperada, o que reduz drasticamente o número de parâmetros necessário para representar a solução. Segundo, “interpretabilidade”: após o treino, podemos ler a expressão analítica da solução diretamente dos pesos remanescentes. A MLP também permite extrair uma expressão analítica, mas a MixFunn é mais interpretável por usar uma rede menor.

A construção do neurônio se dá em dois estágios. Dado um vetor de entrada $\mathbf{x} = (x_1, \dots, x_n)$, o primeiro estágio é uma *pré-mistura quadrática*: além das n entradas, formamos também todos os $n(n-1)/2$ produtos de pares cruzados $x_i x_j$, com $i < j$ (sem repetir índices). Esses $n + n(n-1)/2$ termos são os candidatos a entrar nas funções da camada seguinte.

No segundo estágio, fixamos um conjunto de K funções elementares $\{f_k\}_{k=1}^K$, adotamos

$$\{f_k\} = \{\sin z, \cos z, e^z, \log |z|, z, z^2, 1\}.$$

Para cada função f_k , calculamos um argumento afim z_k a partir dos termos do estágio anterior, e somamos as funções com pesos α_k . Componente a componente,

$$z_k = \sum_{i=1}^n w_i^{(k)} x_i + \sum_{i<j} w_{ij}^{(k)} x_i x_j + b_k, \quad \text{Mix2Funn}(\mathbf{x}) = \sum_{k=1}^K \alpha_k f_k(z_k). \quad (6)$$

Comparando (6) com a forma escalar da MLP (1), a única diferença adicional é o somatório externo em k , que combina respostas de várias funções de ativação em vez de uma só. A pré-mistura quadrática aparece como o termo $\sum_{i<j} w_{ij}^{(k)} x_i x_j$, ausente em (1).

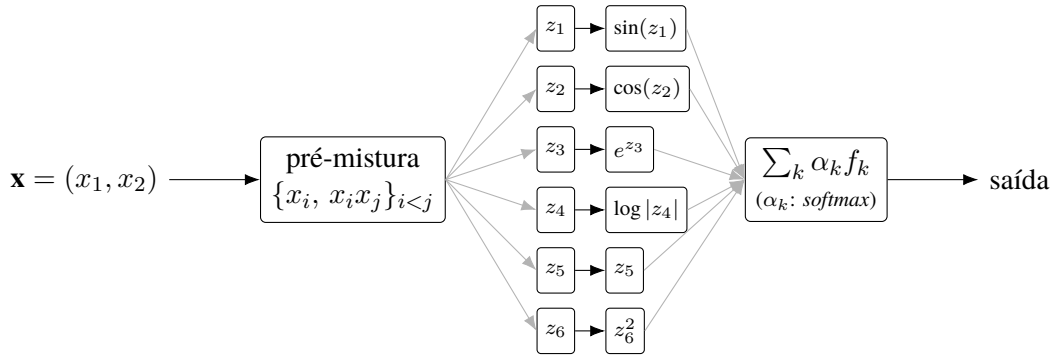


Figura 3: Neurônio Mix2Funn para $\mathbf{x} = (x_1, x_2)$. À esquerda, a pré-mistura forma $\{x_1, x_2, x_1 x_2\}$ a partir das entradas. À direita, cada uma das K funções da base recebe seu próprio argumento afim z_k , e as saídas são combinadas com pesos α_k não-negativos (*softmax*). Para evitar saturação visual, mostramos 6 das 7 funções (a constante 1 é omitida).

Vale notar que a Figura 3 representa um *único* neurônio, na arquitetura, ele substitui o neurônio tradicional da MLP descrito antes. O custo em parâmetros, porém, não é o mesmo: para $n = 2$ entradas, um neurônio MLP tem $n + 1 = 3$ parâmetros, enquanto um neurônio Mix2Funn tem $K(n + n(n-1)/2 + 1) + K = 7 \cdot 4 + 7 = 35$ parâmetros. Cada neurônio Mix2Funn vale, em parâmetros,

cerca de uma dúzia de neurônios da MLP (depende do número de funções escolhidas), mas já vem com a base de funções incorporada.

A *softmax* é uma técnica opcional aplicada à MixFunn que, em alguns problemas, favorece soluções mais simples e interpretáveis ao restringir os pesos α_k a valores positivos. A motivação dessa restrição é que, sem ela, contribuições com sinais opostos podem se cancelar dentro do mesmo neurônio, o que dificultaria identificar quais funções de fato participam da solução. Quando aplicada, a *softmax* recebe um vetor de pesos brutos $\beta = (\beta_1, \dots, \beta_K)$, ajustados livremente pelo otimizador, e devolve

$$\alpha_k = \frac{e^{\beta_k/T}}{\sum_{j=1}^K e^{\beta_j/T}}, \quad \alpha_k \geq 0, \quad \sum_k \alpha_k = 1. \quad (7)$$

A *softmax* é, em essência, uma normalização exponencial: ela transforma qualquer vetor real em uma distribuição de probabilidade. O parâmetro T , chamado “temperatura”, controla o quão concentrada é essa distribuição: T grande achata o vetor em uma distribuição quase uniforme entre as K funções (todas contribuem parecido), enquanto T pequeno acentua o maior peso e suprime os demais (uma função domina).

Camadas empilhadas usam uma matriz de combinação triangular estrita, em que cada neurônio da camada seguinte combina apenas neurônios anteriores. Essa restrição é o que permite, em duas camadas, gerar produtos de pares de funções da base, por exemplo, $\sin(\pi x) e^{-\alpha\pi^2 t}$, solução típica da equação do calor.

A interpretabilidade prometida no início desta seção depende de chegarmos a uma solução em que poucas funções tenham peso significativo. Três técnicas, usadas em conjunto durante o treino, conduzem a rede a esse regime. A primeira é o *annealing* da temperatura: começamos com T alto, para que a rede explore livremente todas as funções com pesos parecidos, e diminuimos T gradualmente ao longo das épocas, o que concentra a *softmax* e força a rede a comprometer-se com uma combinação esparsa. A segunda é o *pruning* por magnitude: ao final do treino, zeramos os pesos cuja magnitude esteja abaixo de um limiar e ajustamos a rede por alguns passos adicionais para que os pesos remanescentes compensem a remoção dos descartados. As funções que sobrevivem ao pruning, ponderadas pelos pesos finais, formam a expressão analítica da solução. A terceira é o *dropout*, que desliga aleatoriamente uma fração dos neurônios durante o treino e impede que a rede dependa de poucos caminhos; é apresentado em mais detalhe na Seção 1.4.

Tanto a PINN (MLP) quanto a MixFunn vêm sendo aplicadas a problemas físicos diversos. A PINN (MLP), originalmente formulada com EDPs como Burgers e Schrödinger não-linear [Raissi 2019], já foi estendida a escoamentos de fluidos reconstruídos a partir de visualização [Raissi 2020] e a problemas eletromagnéticos com interfaces dielétricas [Nohra 2024]. A MixFunn, mais recente, foi proposta inicialmente com problemas em mecânica clássica (osciladores harmônicos amortecido e forçado), fluidos (equação de Burgers) e mecânica quântica (poço quântico infinito) [Farias 2025].

Como reflexo dessa diversidade de aplicações, têm sido publicadas regularmente revisões sistemá-

ticas sobre os avanços e desafios desses métodos [Cuomo 2022], discussões específicas em mecânica de fluidos [Cai 2021], introduções acessíveis para físicos [Miranda 2025], e bibliotecas dedicadas como a DeepXDE [Lu 2021].

Neste trabalho, replicamos casos representativos desse repertório nos dois métodos.

1.4 Generalização

Como qualquer rede neural, a PINN (MLP) e a MixFunn não bastam ajustar bem o erro durante o treino: elas também precisam *generalizar*, ou seja, aprender o padrão dos dados em vez de só decorar valores. Uma rede que generaliza bem responde de modo razoável a pontos que não viu durante o treino. Quando isso falha temos o *overfit*: o erro de treino continua descendo, mas em pontos novos a rede passa a errar, sinal de que a rede ajustou ruído em vez de padrão.

Para identificar overfit a tempo, os dados disponíveis são divididos aleatoriamente em três conjuntos. O conjunto de *treino* é o que entra no loop de otimização: o gradiente do erro é calculado nele e os parâmetros são atualizados. O conjunto de *validação* fica de fora desse loop e serve para ajustar hiperparâmetros (arquitetura, *learning rate*, pesos λ no erro); ele indica se uma escolha de hiperparâmetros generaliza ou só aprende o treino. O conjunto de *teste* é uma camada extra de proteção, ele entra só na avaliação final, para o caso de sobreajustarmos o conjunto de validação durante a escolha de hiperparâmetros. A métrica reportada em todo este trabalho é o erro quadrático médio no conjunto de teste.

Duas técnicas comuns para melhorar a generalização são o *dropout* e a *sub-amostragem*. O *dropout* desliga aleatoriamente uma fração dos neurônios em cada passo do otimizador, forçando a rede a não depender de poucos caminhos. A *sub-amostragem* usa apenas uma fração dos pontos de colocação a cada iteração, introduzindo ruído estocástico que dificulta a memorização. Em ambos os casos o objetivo é o mesmo: impedir que a rede se especialize em particularidades do treino que não representam o padrão geral. Outros métodos de regularização e variantes destes são revisados em [LeCun 2015].

Outro desafio para a generalização é a *instabilidade numérica*. Por exemplo, em arquiteturas como a MixFunn isso aparece nas funções atômicas com singularidade na origem (como $\log(x)$ e $1/x$): adicionamos constantes na sua estrutura para evitar a singularidade (por exemplo, $\log(0,1 + x)$), e essas constantes precisam ser calibradas para a ordem de grandeza do problema. Outro exemplo é quando a ordem de grandeza dos dados de teste difere da dos dados de treino.

2 MATERIAIS E MÉTODOS

Os experimentos deste trabalho usam GPUs especializadas e bibliotecas de aprendizado profundo, como a plataforma CUDA e a biblioteca PyTorch.

Para executar os experimentos deste trabalho, usamos esses recursos diretamente na nuvem: cada treino roda em um *container* efêmero com GPU NVIDIA T4, alocado sob demanda na plataforma

Modal¹. O custo é da ordem de centavos de dólar por minuto de GPU, e o consumo total de cada experimento cabe folgadoamente no *tier* gratuito oferecido pela plataforma. O código é editado localmente, despachado para o *container* sob demanda, e os artefatos (pesos treinados, métricas, *logs*) ficam num volume persistente que a máquina local lê para gerar figuras e análises.

Todo o código está organizado em um repositório *git*², em que cada experimento ocupa uma pasta com quatro *scripts* numerados: pré-processamento, treino, validação e visualização. Os hiperparâmetros são fixados no próprio *script*, sem *flags* de linha de comando, de modo que cada execução fica integralmente registrada no histórico do controle de versão.

A confiabilidade e reprodutibilidade dos resultados se apoiam em três decisões. Primeiro, métodos com componente aleatória (inicialização dos pesos, sorteio dos pontos de colocação, ordem do *batch*) recebem semente fixa em PyTorch e NumPy no início de cada execução; de posse do código e da semente, qualquer código reportado neste trabalho é reproduzível. Segundo, todo conjunto de hiperparâmetros, métricas finais e tempo de execução é gravado em um arquivo *metadata.json* ao lado do modelo treinado, ao final do treino. Terceiro, a matemática de cada método (PINN (MLP) e Mix2Funn) segue exatamente a definida na Introdução; quaisquer técnicas adicionais investigadas aparecem explicitamente nos Resultados como objeto de estudo, não como configuração de *baseline*.

3 RESULTADOS

Este capítulo apresenta quatro problemas de EDPs resolvidos com PINN (MLP) e MixFunn. O foco é uma análise qualitativa dos métodos, para verificar os comportamentos discutidos até aqui. A Seção 3.1 traz uma análise mais profunda dos métodos usando o escoamento de Kovasznay como exemplo. A Seção 3.2 utiliza dados de um aneurisma cerebrovascular humano para calcular o escoamento sanguíneo nele, com o *dataset* ANEUMO [Li 2025]. A Seção 3.3 reúne dois problemas adicionais em regimes físicos distintos, discutidos superficialmente devido à restrição de espaço.

3.1 Navier–Stokes 2D, escoamento de Kovasznay

Definição do problema

O escoamento de Kovasznay [Kovasznay 1948] é uma solução analítica fechada das equações de Navier–Stokes incompressíveis 2D estacionárias, com decaimento exponencial em x modulado por cosseno em y . Adotamos o domínio $\Omega = [-0,5; 1] \times [-0,5; 1,5]$ e $\text{Re} = 40$. O sistema reúne momento em x (Eq.1), momento em y (Eq.2) e continuidade incompressível (Eq.3):

¹<https://modal.com>

²<https://github.com/NicolasRossoni/NNinPhysics>

$$\begin{cases} u u_x + v u_y = -p_x + \text{Re}^{-1}(u_{xx} + u_{yy}), & \text{(Eq.1)} \\ u v_x + v v_y = -p_y + \text{Re}^{-1}(v_{xx} + v_{yy}), & \text{(Eq.2)} \\ u_x + v_y = 0. & \text{(Eq.3)} \end{cases} \quad (8)$$

Aplicando o esquema da Equação (5) ao sistema, a loss não-supervisionada é o resíduo dos três primeiros termos:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{n=1}^N \underbrace{\left(u u_x + v u_y + p_x - \frac{u_{xx} + u_{yy}}{\text{Re}} \right)^2}_{\text{(Eq.1)}} + \underbrace{\left(u v_x + v v_y + p_y - \frac{v_{xx} + v_{yy}}{\text{Re}} \right)^2}_{\text{(Eq.2)}} + \underbrace{(u_x + v_y)^2}_{\text{(Eq.3)}}. \quad (9)$$

A condição de contorno é imposta pela arquitetura, fora da loss [Sukumar 2022]: a saída efetiva u usada no resíduo é

$$u = 1 - e^{\lambda x} \cos(2\pi y) + d(x, y) \tilde{u}, \quad \lambda = \frac{\text{Re}}{2} - \sqrt{\left(\frac{\text{Re}}{2}\right)^2 + 4\pi^2}, \quad (10)$$

onde $\tilde{u}$ é a saída crua da rede, $1 - e^{\lambda x} \cos(2\pi y)$ é a solução analítica de Kovasznay (com $\lambda \approx -0,964$ para $\text{Re} = 40$), e $d(x, y)$ é uma função arbitrária que escolhemos para se anular na fronteira de Ω e crescer suavemente até o valor unitário no interior do domínio. Como $d = 0$ na fronteira, $u = 1 - e^{\lambda x} \cos(2\pi y)$ na fronteira por construção, satisfazendo a condição de contorno sem necessidade de termo de penalidade. v e p são tratados de forma análoga, sendo $v = \frac{\lambda}{2\pi} e^{\lambda x} \sin(2\pi y)$ e $p = \frac{1}{2}(1 - e^{2\lambda x})$ as soluções analíticas correspondentes.

Supervisionado vs. não-supervisionado

Antes do regime não-supervisionado pleno, rodamos uma sonda supervisionada: a rede recebe pares $(x_i, y_i) \mapsto (u, v, p)$ amostrados da analítica, sem o termo da EDP. A loss é definida apenas pelos dados:

$$\mathcal{L}_{\text{sup}}(\theta) = \frac{1}{N} \sum_{n=1}^N (u - u^*)^2 + (v - v^*)^2 + (p - p^*)^2, \quad (11)$$

onde u^*, v^*, p^* são as parcelas analíticas de (10). É um teste de diagnóstico: se a rede não converge sequer com a solução fornecida como referência, ela tampouco convergirá utilizando apenas o resíduo da EDP. A convergência supervisionada confirma que a arquitetura é *capaz* de expressar a solução; só então faz sentido testar se o resíduo é guia suficiente. Se o regime supervisionado falha, a solução não é insistir no não-supervisionado, mas alterar a arquitetura.

Tanto no regime supervisionado quanto não-supervisionado, todas as sementes convergem em ordens de magnitude consistentes: 10^{-3} para PINN (MLP) e 10^{-2} para MixFunn. Por ser mais fácil, existe a possibilidade de o treino supervisionado atingir essa região em menos iterações, mas todos os

Modo + arquitetura	1ª seed	2ª seed	3ª seed	σ
Sup PINN (4×32)	$1,5 \times 10^{-3}$	$1,7 \times 10^{-3}$	$2,5 \times 10^{-3}$	5×10^{-4}
Não-sup PINN (4×32)	$8,4 \times 10^{-4}$	$1,4 \times 10^{-3}$	$1,7 \times 10^{-3}$	4×10^{-4}
Sup MixFunn (3×1)	$3,9 \times 10^{-2}$	$2,4 \times 10^{-2}$	$2,8 \times 10^{-2}$	8×10^{-3}
Não-sup MixFunn (3×1)	$4,6 \times 10^{-2}$	$2,0 \times 10^{-2}$	$7,7 \times 10^{-2}$	3×10^{-2}

Tabela 1: L_{val}^2 (loss) por seed dos modelos treinados contra a solução analítica (supervisionado) e contra o resíduo da EDP (não supervisionado), e σ (desvio padrão amostral). *Para reprodução: ver [Experimento 1 no GitHub](#).*

regimes acabam na mesma vizinhança do espaço de soluções. Por conta disso, a loss é equivalente. Já a variância entre seeds depende da natureza do problema e da estrutura da rede: as três primeiras linhas têm σ pequeno frente ao valor central, mas a MixFunn não supervisionada (quarta linha) varia substancialmente mais, com σ da mesma ordem do valor médio.

Profundidade e largura

Estudamos como o tamanho da rede afeta a loss. Profundidade N (número de camadas) e largura n (número de neurônios por camada) são diretamente proporcionais ao tamanho da rede, e o tamanho controla sua expressividade.

$N \backslash n$	16	32	64
4	$2,0 \times 10^{-3}$	$1,3 \times 10^{-3}$	$1,9 \times 10^{-3}$
6	$2,1 \times 10^{-3}$	$1,5 \times 10^{-3}$	$1,2 \times 10^{-3}$
8	$2,2 \times 10^{-3}$	$1,2 \times 10^{-3}$	$1,0 \times 10^{-3}$

PINN

$N \backslash n$	1	2	3
1	$7,1 \times 10^{-2}$	$7,1 \times 10^{-2}$	$7,0 \times 10^{-2}$
2	$3,1 \times 10^{-2}$	$1,7 \times 10^{-2}$	$3,1 \times 10^{-2}$
3	$2,3 \times 10^{-2}$	$2,3 \times 10^{-2}$	$2,1 \times 10^{-2}$

MixFunn

Tabela 2: L_{val}^2 (loss) por arquitetura para PINN (MLP) (esquerda) e MixFunn (direita). N : número de camadas; n : número de neurônios por camada. Melhor caso em negrito. *Para reprodução: ver [Experimento 1 no GitHub](#).*

Vale destacar que neurônios de MLP e MixFunn não são equivalentes em parâmetros: cada neurônio MixFunn (sem produtos cruzados) tem ~ 35 parâmetros contra $\sim (n + 1)$ por neurônio da MLP, com n entradas (Seção 1.4), de modo que a comparação direta entre arquiteturas de mesmo $N \times n$ é apenas indicativa. Para referência, na Tabela 2 a PINN (MLP) 4×16 tem ~ 915 parâmetros e a 8×64 tem $\sim 29\,500$; a MixFunn 1×1 tem ~ 35 e a 3×3 tem ~ 315 .

Assim como notamos na variância entre seeds (Tabela 1), a loss da maioria das arquiteturas testadas pela PINN (MLP) é parecida: a rede tem mais parâmetros do que o necessário para esse problema, está hiperparametrizada, e ampliá-la a mantém na mesma região do espaço de soluções. É esperado que em detalhe a loss caia um pouco com o tamanho, vemos isso no 8×64 ($L^2 = 1,0 \times 10^{-3}$, melhor da tabela). Mas pensando em eficiência, 4×16 (915 params, $L^2 = 2,0 \times 10^{-3}$) tem essencialmente a mesma loss que 8×64 (29 507 params, $L^2 = 1,0 \times 10^{-3}$), apesar de $\sim 32 \times$ mais parâmetros.

Na MixFunn vemos uma barreira clara em $7,1 \times 10^{-2}$ enquanto há apenas uma camada, independente do número de neurônios. A loss só desce a partir de duas camadas. A razão é estrutural: a solução analítica $1 - e^{\lambda x} \cos(2\pi y)$ contém o produto de duas funções, e a MixFunn só gera produtos por composição de camadas, aumentar neurônios em uma única camada apenas soma funções (hiperparametrização imediata). Existe portanto uma limitação intrínseca: uma rede de uma camada não tem como representar produto, e a loss fica travada nesse patamar.

MixFunn sof. Para mostrar a barreira quantitativamente, treinamos uma variação também apresentada no paper da MixFunn:

$$\text{MixFunn}_{\text{sof}} = \sum_{k=1}^K \alpha_k f_k(z_k) + \sum_{i < j} \alpha_{ij} f_i(z_i) f_j(z_j), \quad (12)$$

onde *sof* é abreviação de *second-order function*, que habilita os produtos cruzados $f_i \cdot f_j$ entre as funções atômicas dentro do mesmo neurônio. É distinto de empilhar uma segunda camada, mas também aumenta a expressividade da rede. Treinada como MixFunn sof 1×1 , atinge $L_{\text{val}}^2 = 1,0 \times 10^{-2}$, abaixo do patamar de $7,1 \times 10^{-2}$ no qual as três configurações de uma camada da Tabela 2 ficaram travadas: comprovamos que a barreira era de expressividade, não de capacidade.

Aqui o diagnóstico é fácil porque conhecemos a solução analítica e podemos isolar o termo crítico. Em problemas sem solução fechada, nunca sabemos a priori se chegamos numa barreira estrutural. Por isso é essencial variar a arquitetura para distinguir hiperparametrização de falta de expressividade. E por isso vale rodar a sonda supervisionada também. *Para reprodução: ver [Experimento 1 no GitHub](#).*

Uma das maneiras de analisar a capacidade de generalização da rede é explorar tanto resultados interpolados (dados de teste dentro da região de treino) quanto extrapolados (dados de teste fora da região de treino). Dentro da região de interpolação, a PINN (MLP) ajusta a intensidade dos picos da função melhor que a MixFunn, o que justifica visualmente a loss menor da PINN observada na Tabela 2. Na extrapolação, ambas divergem progressivamente da analítica, como vemos na Figura 4 em $x = 2$: pelo termo exponencial $e^{\lambda x}$ com $\lambda \approx -0,964$, a analítica decai quase a zero, mas as redes só capturam esse decaimento dentro do intervalo de treino e depois voltam a subir com o sinal invertido.

Como discutido na Seção 1.4, a instabilidade numérica aparece aqui nas funções atômicas da MixFunn ($\log(0,1 + \text{ReLU}(x))$, $\sqrt{0,01 + \text{ReLU}(x)}$): as constantes estão calibradas para a ordem de grandeza do domínio deste experimento; outro domínio exigiria valores diferentes.

Tradicionalmente, os gráficos de loss têm o formato em “taco de hóquei”: descida rápida nos primeiros mil passos e cada vez mais lenta depois. Como o método de otimização busca minimizar a loss, observamos um decaimento ao longo do treino. Os picos visíveis acontecem quando o passo do otimizador é maior que o vale local em que ele estava descendo, mas eventualmente o otimizador encontra o caminho de volta. Aumentar o tamanho do passo torna esses picos mais frequentes e maiores, e em excesso a loss pode não convergir, ajustar o passo é parte do cuidado experimental

Kovasznyai --- campos u com BC Coons, x estendido ate 2,5

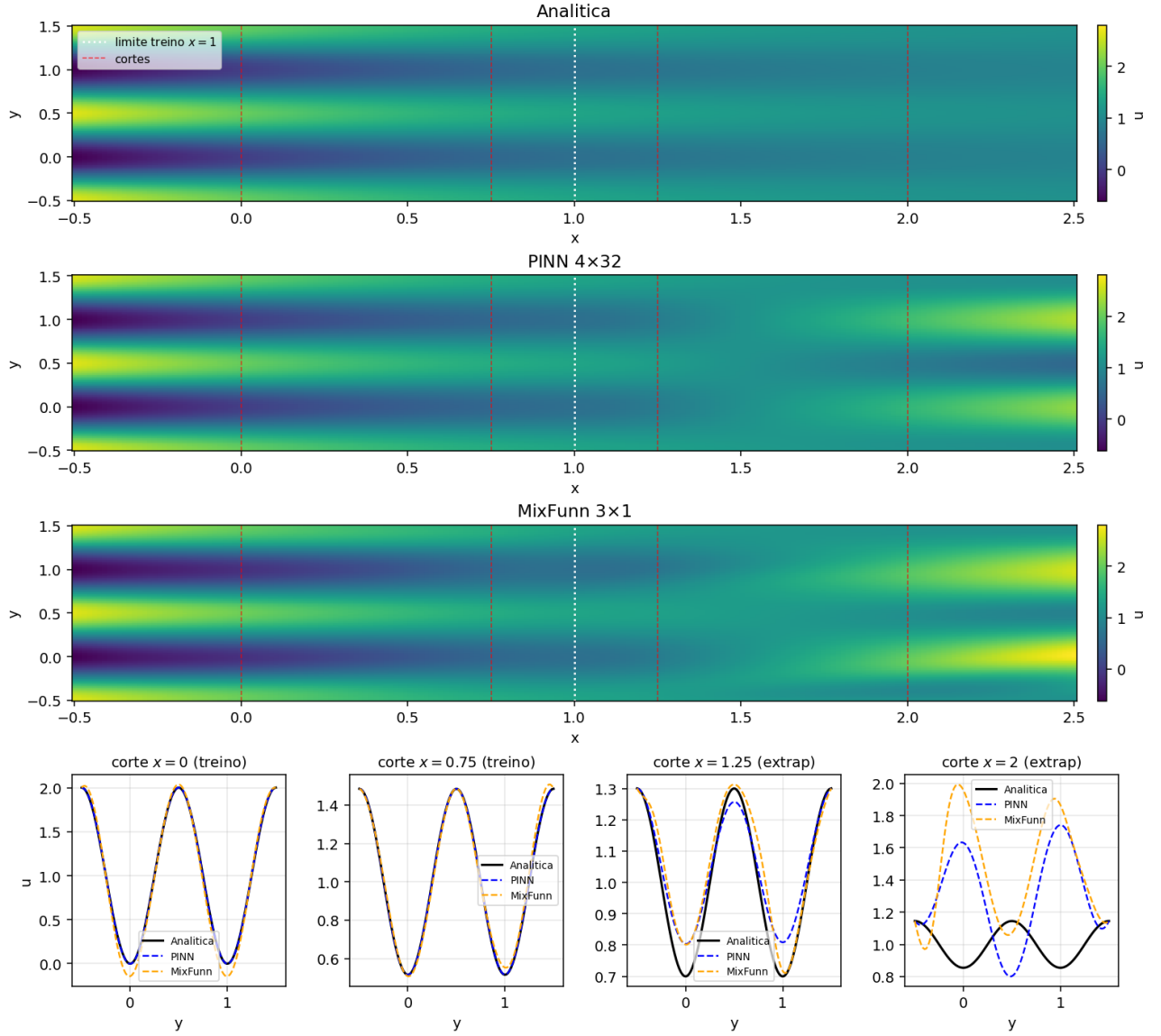


Figura 4: Análise do desempenho da PINN (MLP) 4×32 e da MixFunn_{sof} 1×1 (Tabela 2) dentro do domínio de treino, $x \in [-0,5; 1]$ (interpolação), e fora do domínio de treino, $x > 1$ (extrapolação). Ambas falham em extrapolar; na interpolação a PINN tem desempenho levemente melhor que a MixFunn. *Para reprodução: ver [Experimento 1 no GitHub](#).*

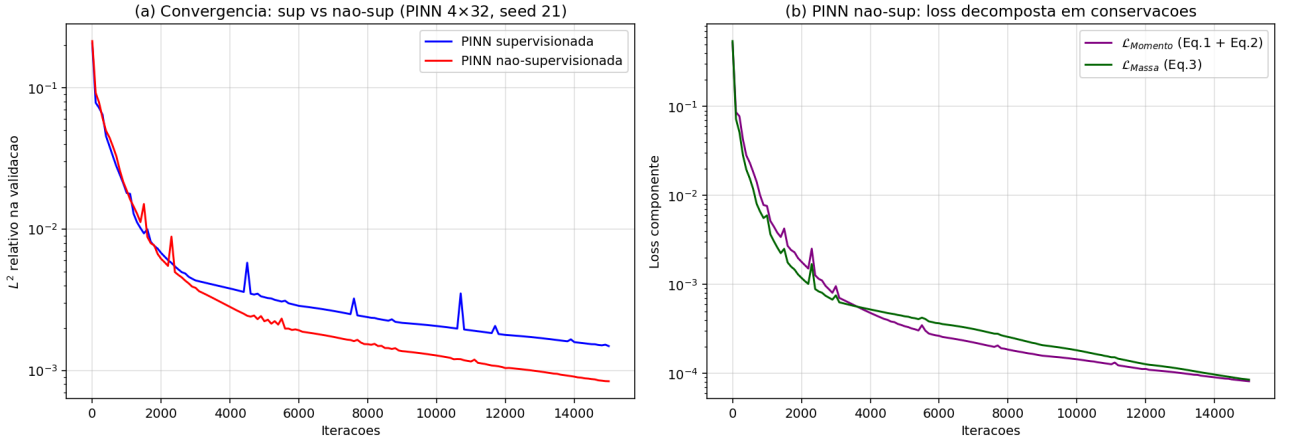


Figura 5: Curvas em escala log para a PINN (MLP) 4×32 . **(a)** Erro contra a solução analítica no conjunto de teste, regime supervisionado (azul) e não supervisionado (vermelho), mesma seed. **(b)** Para a mesma rede não supervisionada do painel (a), decomposição do resíduo da EDP em $\mathcal{L}_{\text{Momento}}$ (Eq.1+Eq.2) e $\mathcal{L}_{\text{Massa}}$ (Eq.3). Os dois gráficos mostram grandezas diferentes. Para reprodução: ver [Experimento 1 no GitHub](#).

durante o treino.

A loss de treino, porém, não é igual à loss de teste: mais iterações não implicam loss de teste menor. Em geral ela satura em algum patamar, e pode até subir em cenários de *overfit* [LeCun 2015], quando a rede passa a aprender ruído dos dados em vez do padrão de fundo. Se estamos treinando uma rede com dados reais que têm ruído (típico do regime supervisionado), é preciso cuidado com hiperparametrização: a rede pode ajustar o ruído desses dados em vez do padrão físico, e a loss de teste piora à medida que a loss de treino continua a descer.

Pelo painel (a) da Figura 5 também notamos que o regime supervisionado não necessariamente aprende mais rápido, apesar de ser um treino “mais fácil”. Pelo painel (b), cada componente da loss de resíduo tem influência diferente no aprendizado. Uma tática comum para colocar um viés no loop de treino é multiplicar cada componente por um peso λ , $\mathcal{L} = \lambda_{\text{Mom}} \mathcal{L}_{\text{Mom}} + \lambda_{\text{Mas}} \mathcal{L}_{\text{Mas}}$, por exemplo dando mais peso à conservação de momento do que à de massa quando a natureza do sistema indica que aquela componente é mais crítica. Os λ não precisam ser constantes: podem variar ao longo do treino, aprendendo primeiro uma componente e depois outra. Esse é mais um aspecto experimental dessas técnicas: variar hiperparâmetros do código é parecido com um experimento empírico de laboratório.

Regularização

Também testamos dois métodos muito comuns de regularização, *dropout* e *sub-amostragem*, em cada uma das duas redes.

A sub-amostragem não fez diferença na PINN (MLP), mas na MixFunn reduziu a loss pela metade; já o dropout piorou a loss em todos os casos, com a única exceção do dropout de 15% na MixFunn. Esse comportamento tem caráter apenas qualitativo: uma caracterização rigorosa exigiria variância de *seed* ampla e maior granularidade nas frações de dropout e sub-amostragem. Ainda assim, reforça

Configuração	PINN (4×32)	MixFunn (3×1)
baseline (sem)	$1,1 \times 10^{-3}$	$4,6 \times 10^{-2}$
dropout 10%	$6,2 \times 10^{-2}$	$8,3 \times 10^{-2}$
dropout 15%	$9,0 \times 10^{-2}$	$3,0 \times 10^{-2}$
sub-amostra 70%	$1,0 \times 10^{-3}$	$2,1 \times 10^{-2}$
sub-amostra 50%	$1,2 \times 10^{-3}$	$2,1 \times 10^{-2}$
combo (drop 10% + sub 50%)	$7,5 \times 10^{-2}$	$2,5 \times 10^{-2}$

Tabela 3: L_{val}^2 (loss) variando dois métodos de regularização (*dropout* e *sub-amostragem*) para a PINN (MLP) 4×32 e a MixFunn 3×1 . Para reprodução: ver [Experimento 2 no GitHub](#).

o caráter empírico da exploração, em que todos os parâmetros importam.

3.2 Aneurisma intracraniano 3D

Após Kovasznay (§3.1), passamos a uma aplicação real para mostrar o desempenho das redes em geometria complexa. Este caso é baseado no *Hidden Fluid Mechanics* (HFM) [Raissi 2018], que infere velocidade e pressão num segmento cerebrovascular a partir de medidas de um escalar passivo (corante) injetado no escoamento. Aqui, baixamos os dados do *dataset* ANEUMO [Li 2025]: uma nuvem de pontos de aproximadamente 8 mm^3 extraída de um paciente real, representando um trecho da árvore vascular cerebral com um aneurisma — um tubo com uma bolsa lateral, o saco aneurismático. Esse tipo de simulação serve, por exemplo, para estimar a pressão dentro do saco e prever risco de ruptura.

Em cada ponto da nuvem, o *dataset* já traz a solução pré-computada pelo CFD comercial Fluent: vetor de velocidade $\mathbf{u} = (u, v, w)$ em m/s e pressão p em Pa, já que o cálculo numérico direto desses campos é complexo. O problema é definido pelas equações de Navier–Stokes incompressíveis, com velocidade zero na parede (definida pela nuvem de pontos do aneurisma), perfil de velocidade fisiológico $\mathbf{u}_{\text{fis}}$ na entrada (extraído do *dataset*) e pressão zero na saída:

$$\begin{cases} (\mathbf{u} \cdot \nabla) \mathbf{u} + \nabla p / \rho - \nu \nabla^2 \mathbf{u} = 0, \\ \nabla \cdot \mathbf{u} = 0, \\ \mathbf{u}|_{\text{parede}} = 0, \quad \mathbf{u}|_{\text{entrada}} = \mathbf{u}_{\text{fis}}, \quad p|_{\text{saída}} = 0. \end{cases} \quad (13)$$

Em um estudo variando a fração de dados ANEUMO usada no treino, apenas o regime 100% supervisionado capturou as características principais do escoamento — o que nos leva a dizer que a rede só conseguiu “aprender” nesse regime.

Isso mostra que as arquiteturas escolhidas *são capazes* de representar a solução: quando guiadas pelos dados, ambas encontram uma solução; o que ainda precisamos melhorar é o algoritmo de busca por resíduo da EDP. Vale notar que o HFM também não opera apenas com o resíduo da EDP — eles usam observações de um escalar passivo (corante) como dado adicional, e mesmo assim o treino de-

Configuração	Supervisionado	MSE
PINN 5×64	0%	$1,80 \times 10^{-2}$
PINN 8×128	0%	$1,78 \times 10^{-2}$
MixFunn 3×1	0%	$1,92 \times 10^{-2}$
MixFunn _{sof} 3×1	0%	$1,83 \times 10^{-2}$
PINN 5×64	25%	$1,32 \times 10^{-2}$
PINN 5×64	50%	$1,33 \times 10^{-2}$
MixFunn _{sof} 2×2	50%	$1,38 \times 10^{-2}$
PINN 5×64	100%	$1,81 \times 10^{-3}$
MixFunn_{sof} 2×2	100%	$3,76 \times 10^{-3}$

Tabela 4: Variações de PINN (MLP) e MixFunn treinadas contra o *dataset* ANEUMO. Linhas em negrito indicam as configurações utilizadas na Figura 6. Para reprodução: ver [Experimento 4 no GitHub](#).

manda semanas em GPU dedicada. Uma alternativa complementar, que propomos como direção de trabalho futuro, seria aprendizado em etapas: treinar a rede primeiro em regimes de Reynolds mais baixo (por exemplo, $Re = 10$), herdar os pesos resultantes como inicialização do treino em regimes mais altos ($Re = 100$, $Re = 300$) e progredir gradualmente até $Re \sim 700$, deixando a rede resolver problemas mais simples antes do mais complexo. Métodos de normalização dos campos físicos ou da loss também podem ajudar. A Figura 6 mostra o *ground truth* ANEUMO e três configurações treinadas: PINN (MLP) supervisionada, MixFunn_{sof} supervisionada e PINN (MLP) 8×128 não-supervisionada. Visualmente, as duas redes supervisionadas recuperam a estrutura geral do escoamento (com perda de detalhe na pressão), enquanto a não-supervisionada não aprendeu o problema.

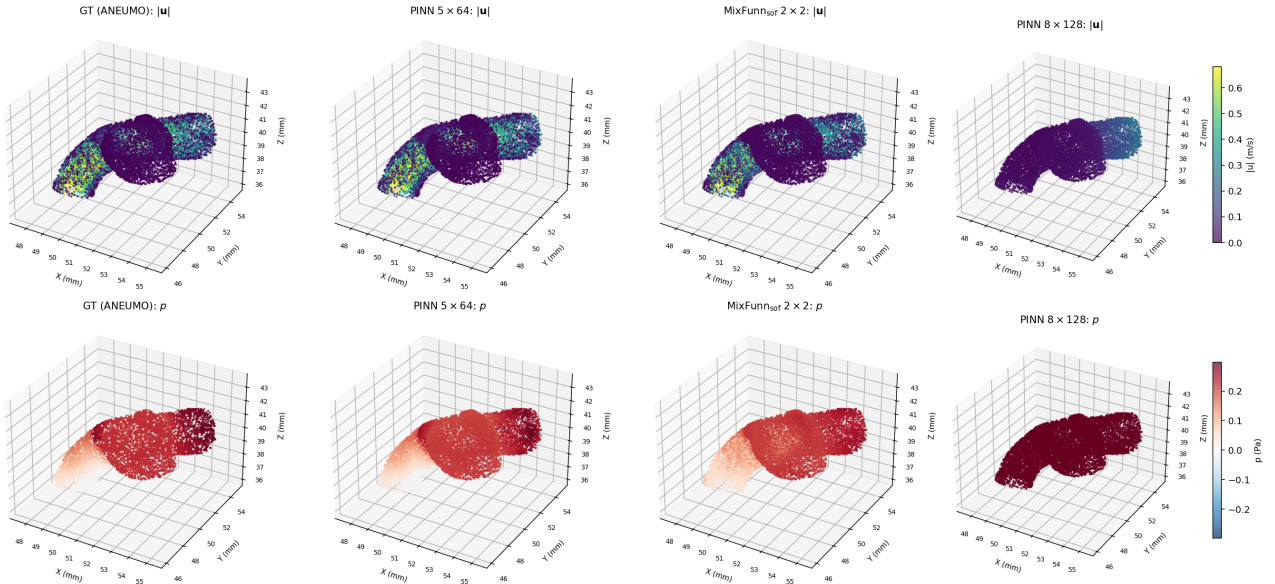


Figura 6: Nuvem 3D colorida por $|u|$ (linha superior) e p (linha inferior). Colunas, em ordem crescente de loss: *ground truth* ANEUMO, PINN (MLP) supervisionada, MixFunn_{sof} supervisionada, PINN (MLP) 8×128 não-supervisionada. Para reprodução: ver [Experimento 4 no GitHub](#).

3.3 Diversidade de EDPs

Os próximos dois problemas são tratados em visão geral qualitativa, demonstrando a aplicabilidade do par PINN (MLP) e MixFunn em regimes físicos distintos. **Todos os resultados a seguir são treinados em regime não-supervisionado**, pelo resíduo da EDP mais condições de contorno, sem usar a referência numérica durante o treino.

Equação de Schrödinger não-linear

A equação de Schrödinger não-linear modela pacotes de onda com auto-interação, comum em óptica não-linear e condensados de Bose–Einstein; a condição inicial $\psi(x, 0) = 2 \operatorname{sech}(x)$ define um sóliton que propaga seu envelope. Como mostra a Figura 7, a PINN (MLP) reproduz o sóliton fielmente, enquanto a MixFunn_{sof} subestima a amplitude do pico central mas recupera bem o decaimento nas extremidades.

$$\begin{cases} i\psi_t + \frac{1}{2}\psi_{xx} + |\psi|^2\psi = 0, \\ \psi(x, 0) = 2 \operatorname{sech}(x), \\ \psi(\pm 5, t) = 0, \end{cases} \quad t \in [0, \pi/2]. \quad (14)$$

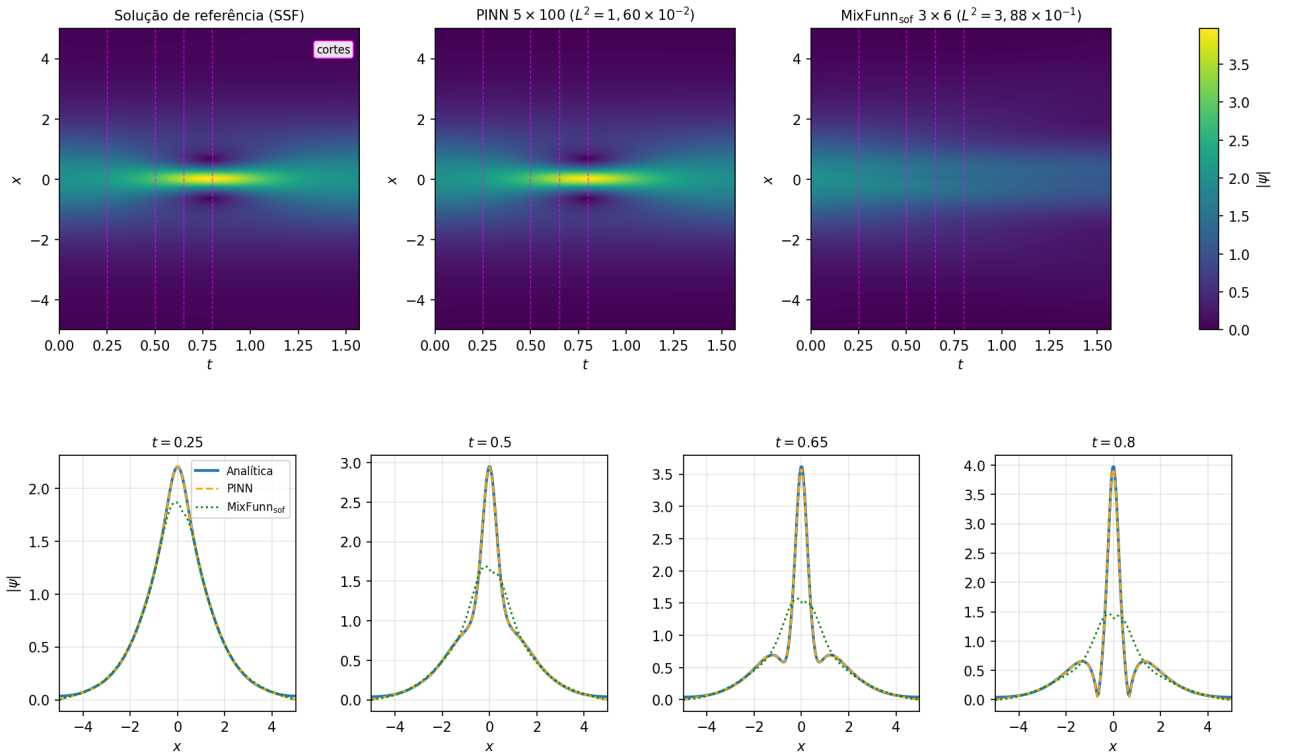


Figura 7: Schrödinger não-linear não-supervisionada, PINN (MLP) e MixFunn_{sof} comparados à solução de referência [Raissi 2019]. Para reprodução: ver [Experimento 6 no GitHub](#).

Espalhamento eletromagnético com disco dielétrico

O problema é magnetostático 2D: um disco de permeabilidade $\mu_r = 3$ em um campo uniforme deforma as linhas de fluxo no entorno, exigindo que a rede satisfaça as equações de Maxwell sem fontes inclusive através da descontinuidade na borda do disco. Como mostra a Figura 8, a PINN (MLP) recupera o padrão perturbado com fidelidade, enquanto a MixFunn_{sof} atinge loss superior, mas reproduz bem a estrutura na vizinhança do disco.

$$\begin{cases} \nabla \times \mathbf{H} = 0, \\ \nabla \cdot (\mu \mathbf{H}) = 0, \\ \mathbf{H}|_{\partial\Omega} = (0, 0, 1), \end{cases} \quad \mu_r = \begin{cases} 3, & \|(x, z) - (0,5, 0,5)\| < 0,2 \\ 1, & \text{caso contrário} \end{cases} \quad (15)$$

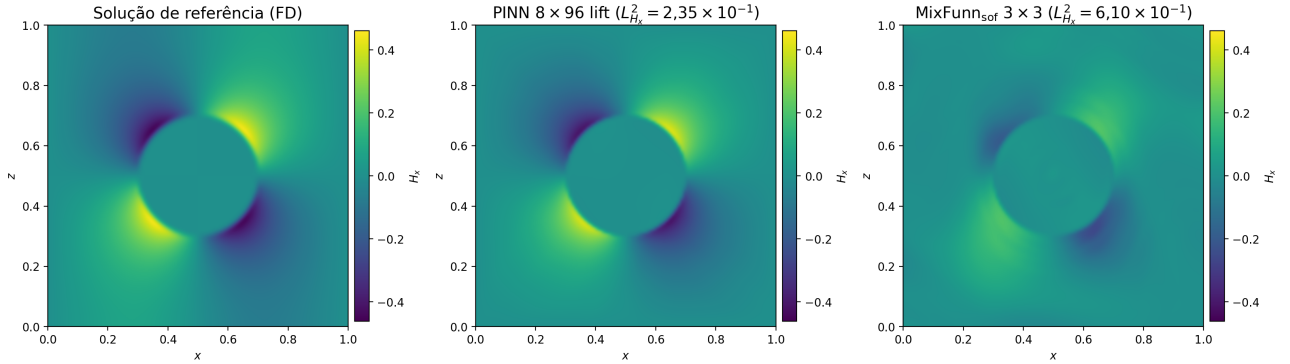


Figura 8: Magnetostática 2D não-supervisionada, componente H_x : PINN (MLP) e MixFunn_{sof} comparados à solução de referência via diferenças finitas (problema de [Nohra 2024]). *Para reprodução: ver [Experimento 7 no GitHub](#).*

Por restrição de espaço, omitimos a discussão da equação de Burgers viscosa, problema replicado conforme [Raissi 2019] ([Experimento 5 no GitHub](#)).

A interpretação dos valores absolutos de loss exige cautela: cada problema tem dimensionalidade e ordem de grandeza próprias, e os números não são diretamente comparáveis entre problemas. Na Schrödinger (Fig. 7) a PINN (MLP) atingiu $L^2 = 1,60 \times 10^{-2}$ contra $3,88 \times 10^{-1}$ da MixFunn_{sof}; no espalhamento eletromagnético (Fig. 8), $L^2_{H_x} = 2,35 \times 10^{-1}$ e $6,10 \times 10^{-1}$, respectivamente. O exame das figuras serve como recurso auxiliar para identificar se a rede aprendeu *features* específicas: a loss pode baixar à custa de especialização em uma região do treino, em detrimento de outras — na Fig. 7, por exemplo, a MixFunn_{sof} reproduz o envelope geral mas subestima o pico central. Um estudo numérico mais rigoroso exigiria variância de *seeds*, varredura do tamanho da rede, teste de métodos de regularização e exploração dos hiperparâmetros, dada a natureza estocástica do treino.

4 CONCLUSÕES E CONSIDERAÇÕES FINAIS

O estudo aprofundado do escoamento de Kovasznay (Navier–Stokes 2D) cobriu múltiplos eixos: variação de arquitetura (profundidade N e largura n), comparação entre regimes supervisionado e não-

-supervisionado e métodos de regularização (*dropout* e sub-amostragem). Em todos esses eixos, tanto a PINN (MLP) quanto a MixFunn convergem em ordens de magnitude consistentes (10^{-3} e 10^{-2} , respectivamente) entre supervisionado e não-supervisionado, com a MixFunn de uma camada apresentando barreira estrutural (loss em $\sim 7,1 \times 10^{-2}$) superada pela variante *second-order function*. A regularização foi explorada qualitativamente: o *dropout* piorou a loss em quase todos os casos e a sub-amostragem reduziu a loss da MixFunn pela metade.

No aneurisma intracraniano 3D, ambas as redes reproduziram a estrutura do escoamento apenas no regime 100% supervisionado (PINN: $1,81 \times 10^{-3}$; MixFunn_{sof}: $3,76 \times 10^{-3}$). Os dois casos complementares (Schrödinger não-linear e magnetostática 2D) mostram a possível diversidade desses métodos em regimes físicos distintos. Conseguimos observar que a PINN (MLP) oferece capacidade expressiva universal, enquanto a MixFunn traz interpretabilidade e arquitetura mais compacta, resolvendo problemas semelhantes com cerca de uma ordem de magnitude menos parâmetros.

As principais dificuldades observadas foram a sensibilidade aos hiperparâmetros e a tendência dos métodos a colapsarem em soluções triviais, especialmente no regime não-supervisionado. No aneurisma intracraniano, por exemplo, o regime não-supervisionado não convergiu para a solução física apesar de testarmos diversas variações de arquitetura e profundidade — exatamente o comportamento de colapso em uma solução trivial mencionado acima, em que a rede atinge uma loss aparentemente baixa correspondendo a um campo aproximadamente nulo, satisfazendo a continuidade incompressível mas não as condições de contorno. No escoamento de Kovasznay, a MixFunn de uma camada apresentou uma barreira estrutural superada apenas pela variante *second-order function*, exemplificando que limitações de expressividade podem ficar mascaradas como problemas de otimização.

REFERÊNCIAS

- 1 RAISSI, Maziar; PERDIKARIS, Paris; KARNIADAKIS, George Em. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019. Disponível em: <https://doi.org/10.1016/j.jcp.2018.10.045>.
- 2 FARIAS, Tiago de Souza; LIMA, Gubio Gomes de; MAZIERO, Jonas; VILLAS-BOAS, Celso Jorge. MixFunn: a neural network for differential equations with improved generalization and interpretability. **arXiv preprint arXiv:2503.22528**, 2025. Disponível em: <https://arxiv.org/abs/2503.22528>.
- 3 LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. Deep learning. **Nature**, v. 521, p. 436–444, 2015. Disponível em: <https://doi.org/10.1038/nature14539>.
- 4 RAISSI, Maziar; YAZDANI, Alireza; KARNIADAKIS, George Em. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. **Science**, v. 367, n. 6481, p. 1026–1030, 2020. Disponível em: <https://arxiv.org/abs/1808.04327>.

- 5 LI, Yuhe et al. ANEUMO: A high-quality dataset for aneurysm-related cerebral hemodynamics. **arXiv preprint arXiv:2505.14717**, 2025. Disponível em: <https://arxiv.org/abs/2505.14717>.
- 6 NOHRA, Michel; DUFOUR, Steven. Physics-Informed Neural Networks for the Numerical Modeling of Steady-State and Transient Electromagnetic Problems with Discontinuous Media. **arXiv preprint arXiv:2406.04380**, 2024. Disponível em: <https://arxiv.org/abs/2406.04380>.
- 7 MIRANDA, Gustavo Cafe de; LIMA, Gubio G. de; FARIAS, Tiago de S. An introduction to neural networks for physicists. **arXiv preprint arXiv:2505.13042**, 2025. Disponível em: <https://arxiv.org/abs/2505.13042>.
- 8 HORNIK, Kurt; STINCHCOMBE, Maxwell; WHITE, Halbert. Multilayer feedforward networks are universal approximators. **Neural Networks**, v. 2, n. 5, p. 359–366, 1989. Disponível em: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- 9 GOLDBERG, David E.; HOLLAND, John H. Genetic algorithms and machine learning. **Machine Learning**, v. 3, p. 95–99, 1988. Disponível em: <https://doi.org/10.1023/A:1022602019183>.
- 10 KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by simulated annealing. **Science**, v. 220, n. 4598, p. 671–680, 1983. Disponível em: <https://doi.org/10.1126/science.220.4598.671>.
- 11 KINGMA, Diederik P.; BA, Jimmy. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014. Disponível em: <https://arxiv.org/abs/1412.6980>.
- 12 KOVASZNAY, L. I. G. Laminar flow behind a two-dimensional grid. **Mathematical Proceedings of the Cambridge Philosophical Society**, v. 44, n. 1, p. 58–62, 1948. Disponível em: <https://doi.org/10.1017/S0305004100023999>.
- 13 BUTCHER, John C. A history of Runge-Kutta methods. **Applied Numerical Mathematics**, v. 20, n. 3, p. 247–260, 1996. Disponível em: [https://doi.org/10.1016/0168-9274\(95\)00108-5](https://doi.org/10.1016/0168-9274(95)00108-5).
- 14 CUOMO, Salvatore et al. Scientific machine learning through physics-informed neural networks: Where we are and what’s next. **arXiv preprint arXiv:2201.05624**, 2022. Disponível em: <https://arxiv.org/abs/2201.05624>.
- 15 CAI, Shengze; MAO, Zhiping; WANG, Zhicheng; YIN, Minglang; KARNIADAKIS, George Em. Physics-informed neural networks (PINNs) for fluid mechanics: A review. **Acta Mechanica Sinica**, v. 37, n. 12, p. 1727–1738, 2021. Disponível em: <https://arxiv.org/abs/2105.09506>.
- 16 SUKUMAR, N.; SRIVASTAVA, Ankit. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. **Computer Methods in Applied Mechanics and Engineering**, v. 389, 114333, 2022. Disponível em: <https://arxiv.org/abs/2104.08426>.
- 17 LU, Lu; MENG, Xuhui; MAO, Zhiping; KARNIADAKIS, George Em. DeepXDE: A deep learning library for solving differential equations. **SIAM Review**, v. 63, n. 1, p. 208–228, 2021. Disponível em: <https://arxiv.org/abs/1907.04502>.